Electronics and
Computer Science

UNIVERSITY OF
Southampton

18. Pipelining (3)

Program Dependencies

- Example:

```
        lw   x1, 0(x7)     # instruction 1
        addi x2, x0, 0     # instruction 2
loop:   addi x2, x2, 1     # instruction 3
        addi x9, x2, 3.    # instruction 4
        blt  x2, x1, loop  # instruction 5
        lw   x4, 0(x8)     # instruction 6
        addi x3, x0, 3     # instruction 7
        add  x3, x3, x4    # instruction 8
        sub  x2, x3, x6    # instruction 9
        and  x4, x6, x2    # instruction 10
```

- Not particularly meaningful. Not well optimised

Data Dependencies

- Flow dependency
  - instruction 8 writes to x3. instruction 9 reads from x3
- Anti-dependency
  - instruction 8 must read from x4 before instruction 10 writes to x4
- Output dependency
  - instructions 3 and 9 both write to x2. instruction 9 cannot complete until instruction 3 has completed, else instruction 5 reads the wrong value
- Anti-dependencies and output dependencies can be eliminated by the compiler – better choice of registers

Pipeline Hazards

- Program dependencies create *hazards* in the pipeline if the dependent instructions could both be in the pipeline at the same time
- The processor must detect these hazards and ensure that the instructions do not interfere
- *Control* or *branch hazards* can occur between instructions that have a control dependency
- *Data hazards* occur when instructions have a data dependency
- *Structural hazards* occur when two instructions try to use the same resource simultaneously

Structural Hazards

- One structural hazard could be in accessing a single memory
  - RISC-V and other RISC cores avoid this by having separate Instruction and Data memories
  - Each is accessed in different stages (see later)
- Might get structural hazards in more complex hardware, e.g. floating-point units

Data Hazards

- A data dependency creates a data hazard when both instructions could be in the pipeline at the same time
  - Read-after-write (RAW) hazard may be caused by a flow dependency, instructions 8 and 9
  - Write-after-read (WAR) hazard may be caused by an anti-dependency, instructions 8 and 10
  - Write-after-write (WAW) hazard may be caused by an output dependency, instructions 3 and 9
- In general, WAR and WAW hazards do not occur in a simple pipelined CPU. As noted, they can be eliminated by the compiler

Pipeline stages

- To understand the hazards, need to be clear what is used in each stage
  - IF – Instruction is read from Instruction Memory; (PC is incremented)
  - ID – Data is read from the register file
  - EX – ALU executes operation
  - MEM – Data is read from, or written to Data memory
  - WB – Data is written to register file

RAW Hazard

- Consider instructions 8 and 9:

```
add  x3, x3, x4 # 8
sub  x2, x3, x6 # 9
```

- Data is written to register x3 in the WB stage of instruction 8
- Data is read from register x3 in the ID stage of instruction 9
- Therefore the WB stage of 8 must come before the ID stage of 9

Pipeline timing RAW hazard

	1	2	3	4	5	6	7	8	9
add	IF	ID	EX	MEM	WB				
sub					IF	ID	EX	MEM	WB

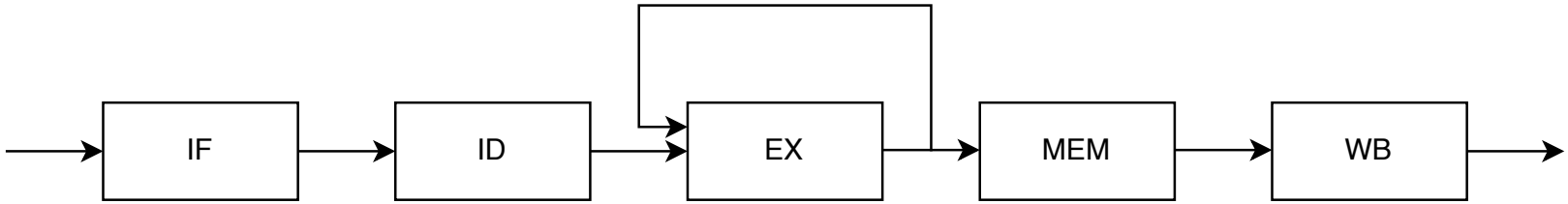
- In other words, the pipeline would have to stall for 3 cycles

	1	2	3	4	5	6	7	8	9
add	IF	ID	EX	MEM	WB				
nop		♣	♣	♣	♣	♣			
nop			♣	♣	♣	♣	♣		
nop				♣	♣	♣	♣	♣	
sub					IF	ID	EX	MEM	WB

Forwarding

- To some extent, this stall might be eliminated by the compiler (different choice of registers), but this is not always effective
- Note, however, that the result of adding x3 and x4 (instruction 8) is available at the end of the EX stage (i.e. before the WB stage completes)
- This result is needed at the start of the EX stage of instruction 9 (i.e. after the IF and ID stages)
- So we can *forward* (in time) this data

Forwarding



- Structurally, a path is created from the output of the EX stage back to the input. (We'll look at the detail of this later.)

	1	2	3	4	5	6
add	IF	ID	EX	MEM	WB	
sub		IF	ID	EX	MEM	WB

- The arrow represents data forwarded from the output of the EX stage of the add instruction to the input of the EX stage of the sub instruction
- No stall!

Forwarding

- Consider this pair of instructions:

```
lw x1, 0(x2)
sub x4, x1, x5
```

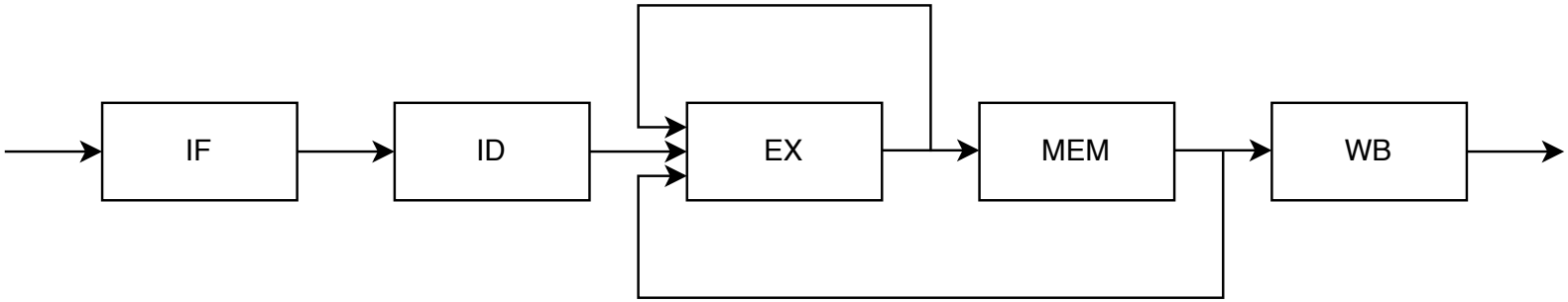
- Similar problem, without forwarding, the pipeline stalls for 3 cycles

	1	2	3	4	5	6	7	8	9
lw	IF	ID	EX	MEM	WB				
sub					IF	ID	EX	MEM	WB

- Now, the data would be available at the end of the MEM stage of `lw` (not the EX stage)

MEM Forwarding

- There is a 2nd forwarding path



- Problem solved?
  - Not quite
  - We still need to stall for one cycle

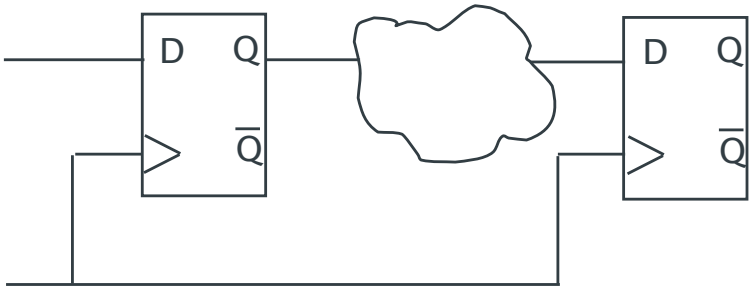
	1	2	3	4	5	6	7
lw	IF	ID	EX	MEM	WB		
nop		♣	♣	♣	♣	♣	
sub			IF	ID	EX	MEM	WB

Can we avoid a stall?

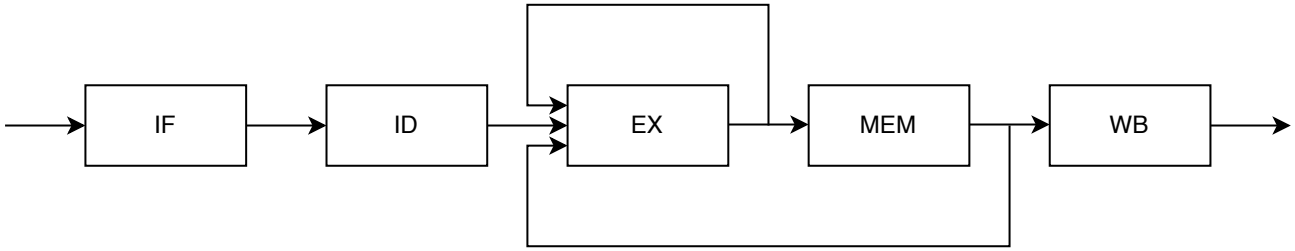
- Not in this case
  - We have to *forward* in time
  - The data from MEM in cycle 4 of lw is forwarded to EX in cycle 5 of sub
  - Without a stall, we would be attempting to send data back in time – a previous cycle
  - We can't send to the same cycle because we would, in effect, be stretching the clock cycle to allow 2 EX operations in the same cycle

Pipeline registers and forward paths

- Each of the pipeline registers (pale blue on next slide) is a clocked register

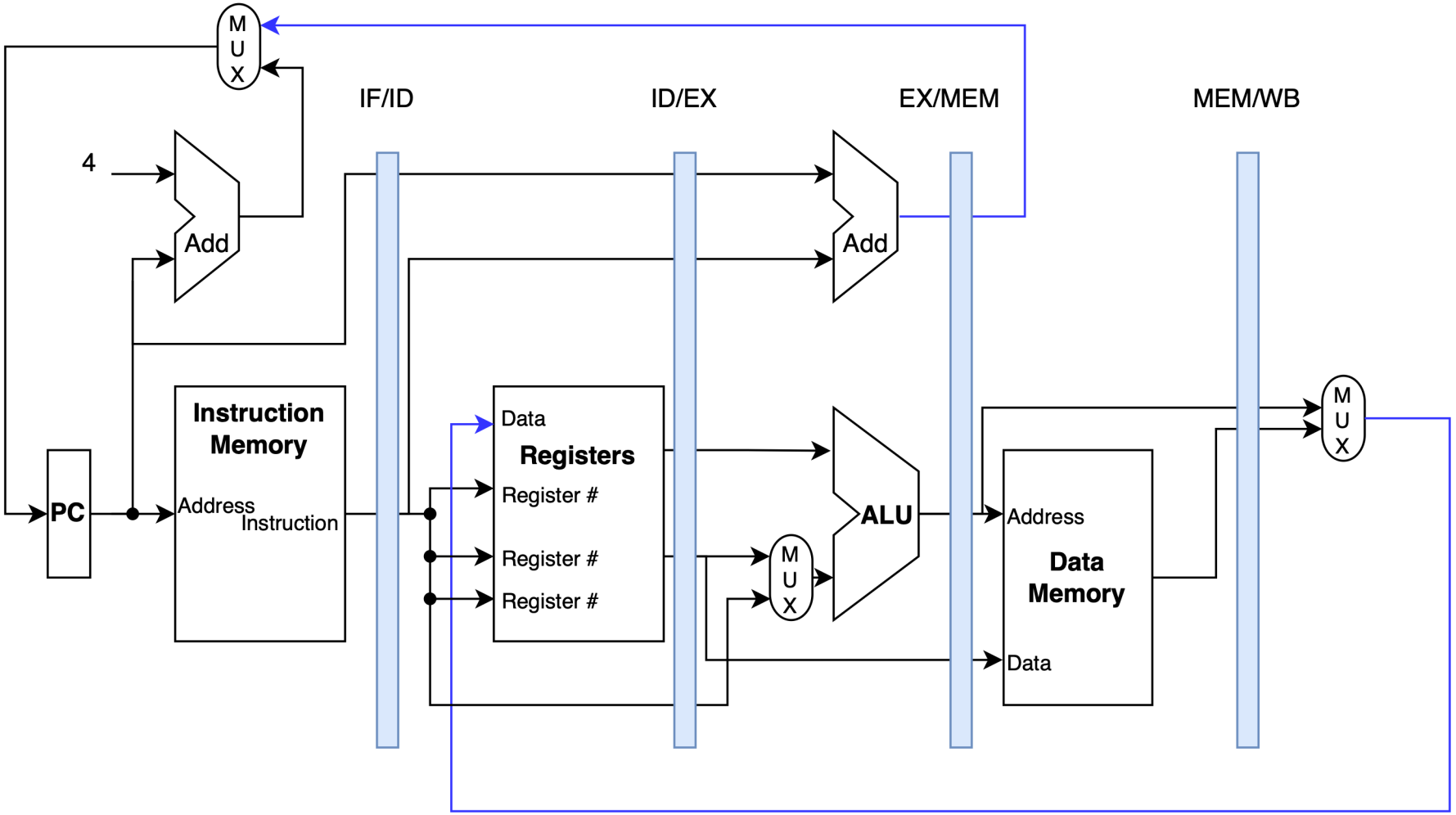


- The forwarding paths start at the outputs of these registers



- The ALU needs MUXes and control logic to use the forwarded data

Pipeline


M
U
X
IF/ID
ID/EX
EX/MEM
MEM/WB
4
Add
Add
Instruction
Memory
Address
Instruction
PC
Data
Registers
Register #
Register #
Register #
M
U
X
ALU
Address
Data
Memory
Data
M
U
X